How-To Library

Informatica

Using the SQL Transformation in an Informatica Developer Mapping

© Copyright Informatica LLC , 2021. Informatica

Abstract

You can run SQL queries against a relational database midstream in a mapping. This article describes how to configure an SQL transformation in a logical data object mapping in the Developer tool.

Supported Versions

- Informatica 9.0.1

Table of Contents

Overview. 2
Input Files. 3
Define the Input Files. 4
Logical Data Object Model. 4
Logical Data Object Mapping. 6
SQL Transformation. 7
Advanced Properties. 7
Ports. 8
SQL Query. 9
Run-time Properties. 9
Filter Transformation. 9
Running the Mapping. 9
Deploy the Mapping. 10

Overview

Your organization receives orders and customer information from multiple sources. The order numbers from one source are alphanumeric. The orders from another source are numeric. You need to combine the orders into one format and generate a company order number for each order.

This article explains how to create a logical data object model to represent a single view of the customer data and the order data. The Order logical data object in the model defines the format of the order numbers. The article also explains how to configure an SQL transformation to generate an order number for each order row in the logical data object mapping.

Complete the following tasks:

1. Define the physical data objects that contain order data.
2. Import the logical data object model that defines a single view of the order data and the customer data.
3. Create a logical data object mapping that maps orders from multiple sources to the Order logical data object. Include an SQL transformation to create an unique order number for each source row.
4. Run the mapping to view results.

Input Files

The input files are flat file order files from different affilates. The content is similar, but the order numbers for one affiliate are alphanumeric. The order numbers from the other affiliate are numeric.

The following figure shows the columns in ReadOrders_Affiliate_A order file:

Order_Id	Customer	Street_Address	City	State	Order_Amt	Product_Id	Product_Desc
A00001	787	38 Busey Street	Redwood City	CA	10.00	555555	Windshield_Wiper
A00002	777	44 Lenolt St	Redwood City	CA	23.50	555577	Door_Handle
A00003	345	41 E Summit Dr	Dardenelle	CA	350.00	450088	Seat_Covers
A00004	120	777 Lakeview Rd	New York	NY	10.00	555555	Windshield_Wiper
A00005	333	910 Castle Hill	Fort Collins	CO	23.50	555577	Door_Handle
A00006	56	216 8th Avenue	San Bruno	CA	350.00	450088	Seat_Covers
A00007	56	216 8th Avenue	San Bruno	CA	75.00	451100	Retractable_Anenna
A00008	187	235 Stoney Rd	Los Angeles	CA	10.00	300000	Wax
A00009	187	235 Stoney Rd	Los Angeles	CA	350.00	450088	Seat_Covers
A00010	200	290 Wave St	San Diego	CA	10.00	555555	Windshield_Wiper

The following figure shows the columns in the ReadOrders_Affiliate_B file:

Order_ID	Product_Number	Description	Customer_ID	Address	City	State	Order_Amt
100000000	123456789	Window Cleaner	100	252 Handy Drive	Foster City	Ca	5.95
100000001	233334545	Octane Booster	121	16 Rut Street	Redwood City	Ca	3.95
100000002	333333567	Rug Shampoo	155	87 Oliver Street	San Mateo	Ca	4.95
100000003	222222456	Oil	183	566 Pamela Lane	Las Vegas	Ca	3.95
100000004	333333567	Rug Shampoo	201	33 VistaPark	Moorpark	Ca	6.95
100000005	111115678	Tire Pump	444	212 Corolla Store	Kimmel	Or	49.99
100000006	444455566	Floor Mat	222	902 Valencia St	San Francisco	Ca	39.99
100000007	123456789	Window Cleaner	222	902 Valencia St	San Francisco	Ca	5.95

Order_ID	Product_Number	Description	Customer_ID	Address	City	State	Order_Amt
100000008	333333567	Rug Shampoo	115	44 Miranda Rd	Palo Alto	Ca	6.95
100000009	233334545	Octane Booster	127	22 Rock Cred Lane	Bakersfield	Ca	3.95

Define the Input Files

Create physical data objects to define the input files and define the path to each file. Define a location that the Data Integration Service can access.

The following figure shows the physical data object in the workspace and the run-time properties for Affiliate A order file:



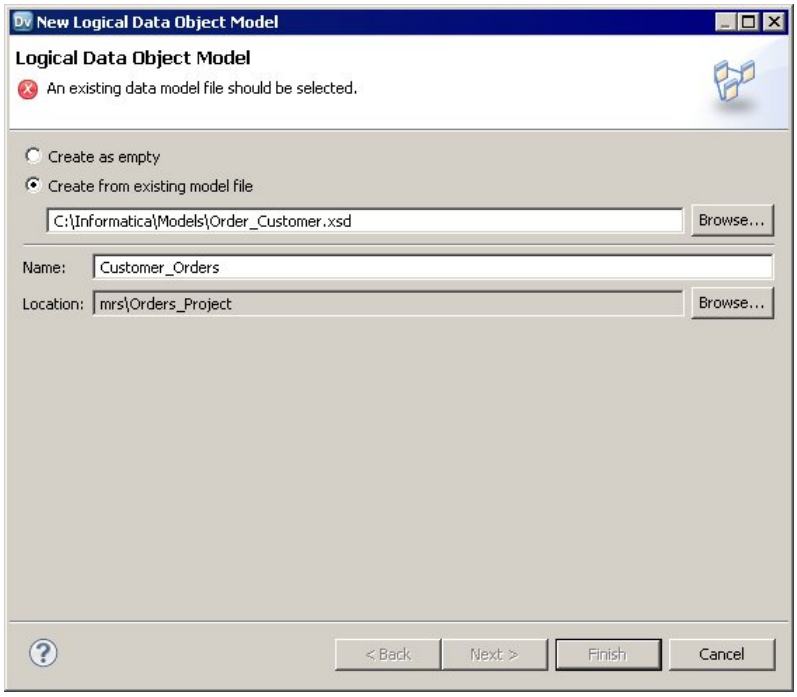
Configure the source file name and source file path in the run-time properties. To access run-time properties, click the Output transformation and click the Runtime tab in the Properties view.

Logical Data Object Model

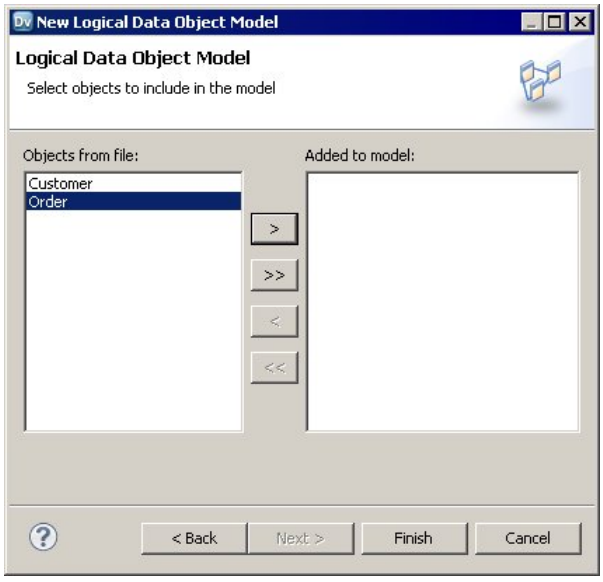
The logical data object model contains a unified view of the customers and orders from Affiliate A and Affiliate B. The logical data model contains the Customer logical data object and the Order logical data object. You can import a logical data object model from an .xsd file that you created in a modeling tool such as ERwin. Or, you can manually create a logical data object model in the Developer tool.

For this example, import the logical data object model from Order_Customer.xsd:

1. Click **File** > **New** > **Logical Data Object Model.**
   The New Logical Data Object Model dialog box appears:



2. Select **Create from Existing Model File**.
3. Browse for the location of the Order_Customer.xsd file.
4. Enter a name of the logical data object model. The name is Customer_Orders.
5. Browse for the location of your project in the repository. The default is the current location.
6. Click **Next**.
   A dialog box appears and lists the objects you can import from the .xsd file.



7. Select the Customer and Order objects from the list.
   The Developer tool creates both logical data objects.

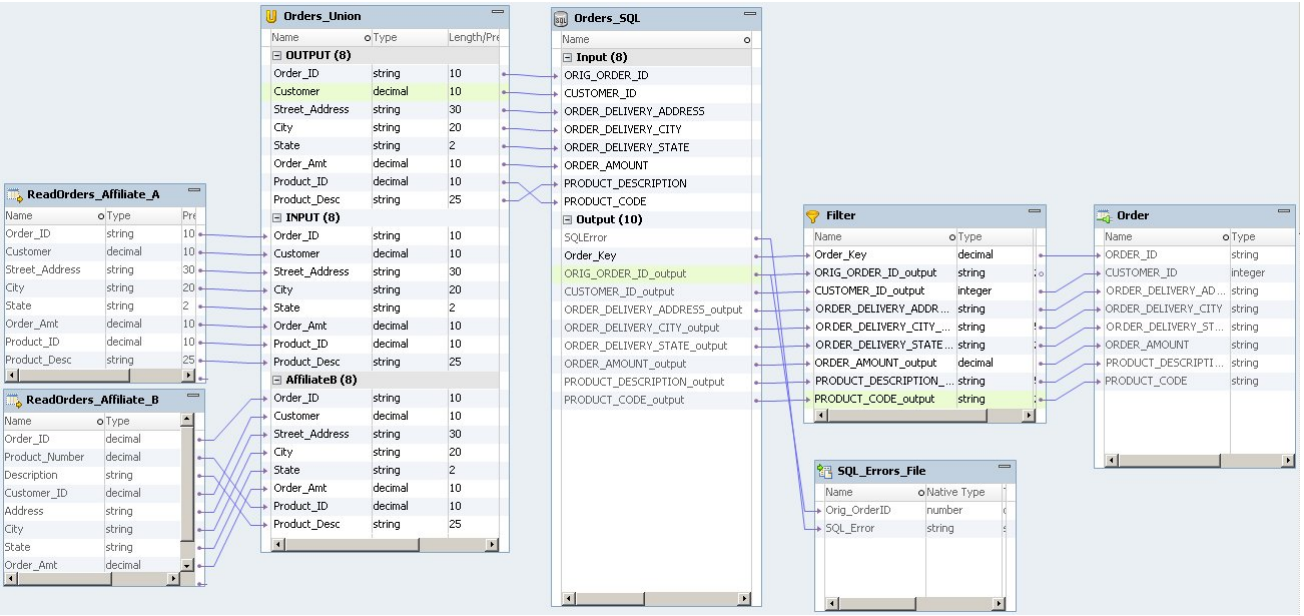
Logical Data Object Mapping

The logical data object mapping describes how you access data through a logical data object. A data object mapping might contain transformations that cleanse or change the data.

For this example, create a logical data object mapping to map the orders from multiple sources to the Order logical data object.

1. Right-click the Order logical data object in the **Navigator**. Choose **Map Logical Data Object**.
   The New Data Object Mapping dialog box appears.
2. Choose the Read **Type.**
3. Enter a name for the mapping.

The following figure shows a logical data object mapping that creates a single view of the orders:



The mapping contains the following objects:

ReadOrders_Affiliate_A

The input file of orders from affiliate A.

ReadOrders_Affiliate_B

The input file of orders from affiliate B.

Orders_Union

Union transformation that combines the rows from the input files. The Union transformation returns an alpha numeric order number for all orders.

Orders_SQL

SQL transformation that assigns an order number to each row and increments a sequence number in a relational table.

Filter

Filter transformation that filters rows that have NULL order numbers from the mapping.

SQL_Errors_File

Flat file that receives the original customer order number and the SQL error messages that the database might return. The SQL_Errors file has the following structure:

Column	Size	Description
Orig_OrderID	20	Original order number
SQL_Error	4096	Error message

The SQL_Errors file receives two rows for each order because the SQL transformation returns a row for each SQL statement. Each row contains NULL in the SQL_Error column unless the database returns an error message for the SQL statement.

Order

Logical data object that contains the single view of the customer orders. Order receives one row for each order.

SQL Transformation

The SQL transformation processes two SQL statements midstream in the mapping. The transformation retrieves a sequence number from a table and returns it in an output port. The SQL transformation increments the sequence number in the database table.

The Sequence_Number table contains the sequence number that the Data Integration Service applies to each customer order. The Sequence_Number table is in an Oracle database. The sequence number becomes the company order number in the logical data object.

Configure the following properties in the SQL transformation views:

- Advanced Properties. Configure the database type. The column datatypes depend on what database type you choose.
- Ports. Define the pass-through ports and the output port that receives the order number from the Sequence_Number table.
- SQL. Define the SQL query to retrieve the sequence number and to increment the number in the database table.
- Runtime. Choose a connection to the database that contains the sequence numbers.

Advanced Properties

When you configure advanced properties, the Developer tool changes the ports or port datatypes in the SQL transformation. Configure advanced properties before you configure the ports.

Configure the following properties in the Advanced tab of the Orders_SQL transformation:

Database type

The database type is Oracle. When you define ports, you can choose port datatypes that are applicable for Oracle.

Continue on Error Within Row

Disable. Stop processing if an error occurs in the row.

Include Statistics as Output

Disable. The Orders_SQL SQL transformation updates one row in the Sequence_Number table for each row that the SQL transformation reads. You might enable the Include Statistics as Output when the SQL statement inserts, updates, or deletes multiple table rows for each input row.

Query

The Advanced Properties view shows the SQL query that you define in the SQL Editor. Define the query in the SQL view instead of the Advanced Properties view.

Note: The Connection Type and SQL Mode properties are read-only. The connection type is static. The SQL transformation connects one time to the database with the connection object that you configure in the SQL transformation run-time properties. The SQL Mode is query. The SQL transformation runs a query that you define in the transformation.

Ports

The output port returns the value from the SQL query. The pass-through ports pass the data in the input row through the transformation. The SQL transformation does not change the data in pass-through ports.

The following figure shows the SQL transformation ports:

Ports | SQL | Runtime | Advanced

	Name	Type	Native Type	Precision	Scale	Default Value	Description	Copy to Output
	Input (8)							
1	ORIG_ORDER_ID	string	nvarchar2	20	0			☑
2	CUSTOMER_ID	integer	number(p,s)	10	0			☑
3	ORDER_DELIVERY_ADDRESS	string	varchar	100	0			☑
4	ORDER_DELIVERY_CITY	string	varchar	50	0			☑
5	ORDER_DELIVERY_STATE	string	varchar	20	0			☑
6	ORDER_AMOUNT	decimal	number(p,s)	18	2	0		☑
7	PRODUCT_DESCRIPTION	string	varchar2	500	0			☑
8	PRODUCT_CODE	string	varchar2	20	0			☑
	Output (10)							
1	SQLError	string		4096	0			☐
2	SEQUENCE_NO	decimal	number(p,s)	18	0			☐
3	ORIG_ORDER_ID_output	string	nvarchar2	20	0			☐
4	CUSTOMER_ID_output	integer	number(p,s)	10	0			☐
5	ORDER_DELIVERY_ADDRESS_out...	string	varchar	100	0			☐
6	ORDER_DELIVERY_CITY_output	string	varchar	50	0			☐
7	ORDER_DELIVERY_STATE_output	string	varchar	20	0			☐
8	ORDER_AMOUNT_output	decimal	number(p,s)	18	2			☐
9	PRODUCT_DESCRIPTION_output	string	varchar2	500	0			☐
10	PRODUCT_CODE_output	string	varchar2	20	0			☐

All of the ports are pass-through ports except the SQLError and Sequence_No output ports.

When you create the SQL transformation, the Developer tool creates the SQLError port. The SQLError port returns SQL errors from the database. You cannot delete this port.

The Sequence_No output port returns the sequence number value from the Sequence_Number table.

When you configure a query SELECT clause, the number and the order of the output ports must match the number and order of the fields in the SELECT clause. The native type for the port must match the datatype of the column in the database.

When the number of output ports is more than the number of columns in the SELECT clause, the extra ports receive a NULL value. When the number of output ports is less than the number of columns in the SELECT clause, the Data Integration Service generates a row error.

To create a pass-through port, drag the ports from the Union transformation to the SQL transformation. In the Ports view, select Copy to Output. The Developer tool creates the output ports and adds the suffix "_output" to each port name.

SQL Query

Access the SQL Editor in the SQL view.

Configure the following query in the SQL Editor:

```
SELECT SEQUENCE_NO from SEQUENCE_NUMBER;
UPDATE SEQUENCE_NUMBER
Set SEQUENCE_NO =  SEQUENCE_NO + 1 ;
```

The following figure shows rows that the SQL transformation generates:



	SQLError	SEQUENCE_NO	ORIG_ORDER_ID...	CUSTOMER_ID...	ORDER_DELIVE...	ORDER_DELIVE...	ORDER_DELIVE...	ORDER_AMOU...	PRODUCT_DES...	PRODUCT_COD...
1	<null>	222	100000000	100	252 Handy Drive	Foster City	Ca	5.95	Window Cleaner	123456789
2	<null>	<null>	100000000	100	252 Handy Drive	Foster City	Ca	5.95	Window Cleaner	123456789
3	<null>	223	100000001	121	16 Rut Street	Redwood City	Ca	3.95	Octane Booster	233334545
4	<null>	<null>	100000001	121	16 Rut Street	Redwood City	Ca	3.95	Octane Booster	233334545
5	<null>	224	100000002	155	87 Oliver Street	San Mateo	Ca	4.95	Rug Shampoo	333333567
6	<null>	<null>	100000002	155	87 Oliver Street	San Mateo	Ca	4.95	Rug Shampoo	333333567
7	<null>	225	100000003	183	566 Pamela Lane	Las Vegas	Ca	3.95	Oil	222222456
8	<null>	<null>	100000003	183	566 Pamela Lane	Las Vegas	Ca	3.95	Oil	222222456

The SQL transformation returns two rows for each input row. The transformation returns one row for the SELECT statement and another row for the UPDATE statement in the query.

When a query contains a SELECT clause, the SQL transformation returns the data in the output ports, the pass-through ports, and the SQLError port.

When a query contains an UPDATE clause, the query returns no data from the database. The SQL transformation returns data to the SQLError port and the pass-through ports. Output ports return NULL values.

The SQLError port returns NULL when no SQL errors occur.

Run-time Properties

Choose a database connection in the Runtime view.

The SQL transformation connects to the database with the database connection object that you choose. The database type in the connection must be the same database type that you configure in the Advanced Properties view.

Filter Transformation

The Filter transformation removes the rows that contain NULL sequence numbers.

The Filter transformation has the following expression in the Filter view:

```
SEQUENCE_NO > 0
```

The Filter transformation returns one row for each order. Each row has a unique sequence number. The sequence number becomes the company order number in the logical data object.

Running the Mapping

To run the mapping, click **Run** > **Mapping**. When you run the mapping, the Filter transformation passes the following rows to the Order logical data object:



	SEQUENCE_NO	ORIG_ORDER_I...	CUSTOMER_ID...	ORDER_DELIVE...	ORDER_DELIVE...	ORDER_DELIVE...	ORDER_AMOU...	PRODUCT_DES...	PRODUCT_COD...
1	262	100000000	100	252 Handy Drive	Foster City	Ca	5.95	Window Cleaner	123456789
2	263	100000001	121	16 Rut Street	Redwood City	Ca	3.95	Octane Booster	233334545
3	264	100000002	155	87 Oliver Street	San Mateo	Ca	4.95	Rug Shampoo	333333567
4	265	100000003	183	566 Pamela Lane	Las Vegas	Ca	3.95	Oil	222222456
5	266	100000004	201	33 VistaPark	Moorpark	Ca	6.95	Wax	333333567
6	267	100000005	444	212 Corolla Stre...	Kimmel	Or	49.99	Tire Pump	111115678
7	268	100000006	222	902 Valencia St...	San Francisco	Ca	39.99	Floor Mat	444455566
8	269	100000007	222	902 Valencia St...	San Francisco	Ca	5.95	Window Cleaner	123456789
9	270	100000008	115	44 Miranda Road	Palo Alto	Ca	4.95	Rug Shampoo	333333567

The sequence number becomes the company order number in the logical data object.

Deploy the Mapping

You might want to make the combined order data available to members of your organization. You can create an SQL data service and deploy it as an application. For more information, see the *Informatica Developer User Guide*.

Author

Ellen Chandler

Acknowledgements